



中山大學
SUN YAT-SEN UNIVERSITY

多模态大模型原理与应用课程报告

基于视觉理解与自动操作的智能浏览器助手

姓名 _____

学号 _____

学院 _____ 计算机学院

专业 _____ 计算机科学与技术

目录

1	应用背景	2
2	任务定义	2
一	相关工作调研	3
1	多模态网页 Agent	3
2	视觉定位与混合感知	3
3	相关开源项目	4
二	系统方案设计	4
1	SoM 视觉输入模式	4
2	HTML 文本输入模式	6
三	实验评测与分析	7
1	HTML 输入模式下的性能测评	7
2	SoM 输入模式下的性能测评	8
四	参考文献	9

1. 应用背景

随着互联网的快速发展，网页的样式和信息量都在经历日新月异地增长，为人们完成特定的网页任务带来了挑战。在多模态大模型技术成熟的背景下，基于 VLM/LLM 技术实现的多模态网页 Agent 能够实现辅助和某种程度地替代人类完成对应的网页任务，具有广阔的应用前景。Web Agent 可以作为自动化测试工具，辅助软件开发团队进行前端的冒烟测试与回归测试。Web Agent 也可以作为智能个人助理，帮助个人用户执行复杂任务，降低网页任务的操作成本。除此以外，Web Agent 可以实现复杂数据的收集与分析、改进针对障碍人士的辅助功能等任务。

2. 任务定义

我们计划实现一个能够理解自然语言指令，并通过浏览器自主完成复杂任务的智能代理。用户将网页任务目标作为输入，在执行任务的每一步中获取网页的 Accessibility

Tree 并根据网页截图进行 SoM 标记，将网页截图与 SoM 输入多模态大模型，生成对应的指令，并使用 playwright 执行并等待页面反馈、进行任务的下一步。基于任务导向，我们计划实现：

- 一个 SoM 内容标注器，根据网页截图和 Accessibility Tree 生成元素边界框和对应的 id，提高模型的准确率
- 一个基于 VLM 的 Web Agent，根据用户输入进行任务的分步计划、在每一步根据网页截图信息和 SoM 标注生成对应的网页操作指令
- 一个 playwright 执行器，解析 Agent 输出的执行指令并执行，刷新网页进行下一步任务

一 相关工作调研

本课题相关工作涉及多模态 GUI 代理、网页自动化框架、任务规划与视觉定位等方向，以下列举代表性论文与开源项目。

1. 多模态网页 Agent

- **WebVoyager** [1]: 提出基于 GPT-4V 的端到端网页 Agent，在真实网站上通过截图和 HTML 辅助实现高成功率，验证了大视觉模型在网页导航中的可行性。
- **Fara-7B** (Microsoft Research) [2]: 一个 7B 参数的多模态模型，结合 Magentic-UI 框架，使用 Playwright 执行操作，在控制复杂度的同时实现良好性能，是本课题的核心参考。
- **WebAgent** 综述 [3]: 对基于 LLM 的网页自动化 Agent 进行了系统分类，总结了规划、感知、执行的典型架构。

2. 视觉定位与混合感知

- **SoM (Set-of-Mark)** [4]: 通过为可交互元素叠加数字标记，将 GUI 定位任务转化为标记选择任务，显著提升 VLM 在界面上的 grounding 准确率。本系统将 SoM 作为预处理模块，为 VLLM 提供可直接引用的标记索引。

- **R-VLM** [5]: 提出 IoU 感知的目标函数和缩放区域候选框, 将 GUI 元素定位精度大幅提升, 在 Mind2Web 基准上获得高准确率, 为改进视觉定位提供了理论支持。
- **WebGUM** [6]: 证明同时利用 HTML/可访问性树和视觉信息比纯视觉或纯 DOM 方法更有效, 为本课题混合感知策略提供了实验依据。

3. 相关开源项目

- **Playwright** [7]: 现代浏览器自动化框架, 提供跨浏览器、可靠的选择器和等待机制, 本课题的执行层核心。
- **OpenClaw / Nanobot** [8]: 开源的多模态 Agent 实现, 展示了从截图到操作的简易流水线, 但缺乏精心设计的错误恢复和混合感知。
- **SMART-LLM** [9]: 多 Agent 协作框架, 用于复杂任务规划, 可借鉴其任务分解与验证机制。

二 系统方案设计

1. SoM 视觉输入模式

SoM (Set-of-Mark) 模式是系统的主要运行模式。该模式将网页截图与可交互元素的视觉标记相结合, 使 VLM 能够直观地看到页面上的可操作元素, 并通过引用标记编号来精确指定操作目标。其整体架构如图1所示, 遵循 ReAct (Reasoning + Acting) 范式, 每步操作分为感知 (Perceive)、推理 (Reason)、执行 (Act) 和验证 (Verify) 四个阶段。

感知阶段 (Perceive) 负责将当前网页状态转化为 VLM 可理解的表示。首先通过 Playwright 截取当前页面的全页面截图, 同时通过 ElementExtractor 模块使用 CSS 选择器 (覆盖 `<button>`、`<a>`、`<input>` 等原生交互标签, 以及 `[role]`、`[aria-label]`、`[data-a11y]` 等富交互属性) 提取页面中所有可交互的 DOM 元素。对于现代 SPA 框架中无任何 HTML 属性的按钮, 通过检测 `cursor: pointer` 计算样式作为补充识别手段。元素经过可见性、尺寸 (最小 4px^2)、disabled 和 pointer-events 过滤后, 由 SoMMarker 模块在截图上绘制带编号的彩色矩形边框 (12 色调色板)。同时 DOMBridge 维护 SoM 编号到 CSS 选择器及 Playwright 定位器的双向映射, 使后续执行阶段能将 VLM 输出的元

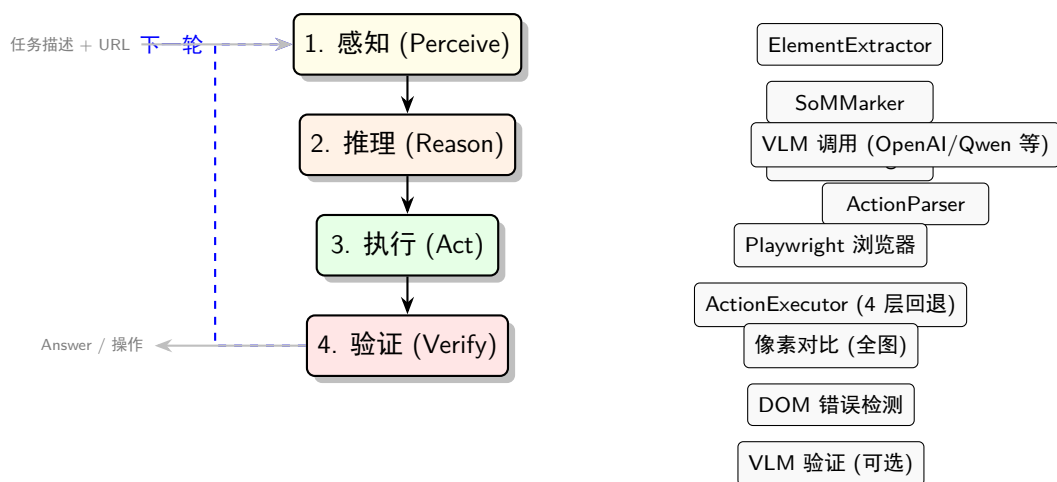


图 1: SoM 模式的 ReAct 循环架构

素编号解析为可点击的 DOM 元素。对于旧版 QQ 邮箱等使用 frameset 架构的网页，感知阶段会遍历所有 `<iframe>` 子文档并修正坐标偏移，确保标注位置正确。

推理阶段 (Reason) 将 SoM 标注后的截图以 base64 JPEG 格式连同任务描述、当前页面 URL/标题、历史操作记录（含验证成功/失败标记）和可交互元素文字列表（编号、标签、文字内容、aria-label）一并发送给 VLM。VLM 返回的结构化 JSON 包含操作类型（click/type/scroll/press/goto/answer/fail）、目标元素编号和操作值，同时还包含对已完成步骤的总结和后续计划，帮助模型在跨步骤任务中保持连贯性。

执行阶段 (Act) 由 ActionExecutor 负责将 VLM 决策转化为浏览器操作。点击操作采用 4 层回退机制：优先使用 Playwright 原生点击（产生 `isTrusted=true` 的真实事件），失败后依次尝试 JS MouseEvent 派发、基于元素真实链接的 goto 导航（绕过 Bing 等网站的点击拦截）、坐标中心点的 `elementFromPoint` 命中，最后回退至屏幕坐标点击。对于 TYPE 操作，强制要求先点击输入框获取焦点后再输入。执行完成后检测 URL 变化和新增标签页，自动切换到最新打开的标签页。

验证阶段 (Verify) 对操作效果进行判断。首先通过 PIL 的 `ImageChops.difference` 进行全图像素级对比，差异像素少于 0.05% 时直接判定为无效操作。随后扫描 DOM 中的错误弹窗和成功提示（QQ 邮箱的 `xmail-ui-dialog` 等）。若以上检测均未触发，可选调用 VLM 进行视觉对比验证。验证失败的步骤在下一轮推理时以 [NO EFFECT] 标记显示在历史记录中，引导模型尝试替代方案。

2. HTML 文本输入模式

HTML 模式是为 Mind2Web 数据集评测而引入的纯文本输入变体。其设计目标是让同一个 Agent 框架能够在不依赖截图的情况下，仅通过 HTML 元素文本列表完成网页操作评估。由于 Mind2Web 数据集预先标注了每个步骤的正负候选元素及其 `backend_node_id`，HTML 模式不需要截图和 SoM 标注，改为直接从数据集读取候选列表并格式化为编号文本供 LLM 选择。

HTML 模式与 SoM 模式共享完全相同的 Agent 循环和 Executor，差异仅在于 Perceptor 和 Prompt。如图2所示：

- **感知层**：由 HTMLPerceptor 替代 SoMPerceptor，离线评估时读取 Mind2Web 候选数据，实时运行时可回退至 ElementExtractor 提取真实页面元素，同时提取 `body.innerText` 作为页面内容摘要供 LLM 理解当前页面状态。
- **推理层**：Reasoner 自动检测 `perception.screenshot` 是否为空，若为空则切换至 HTML 文本 Prompt，将元素列表、页面文本、历史记录和当前计划格式化为纯文本发送给 LLM。
- **验证层**：HTML 模式下无截图，验证功能被简化为 URL 变化检测和 DOM 错误弹窗扫描。

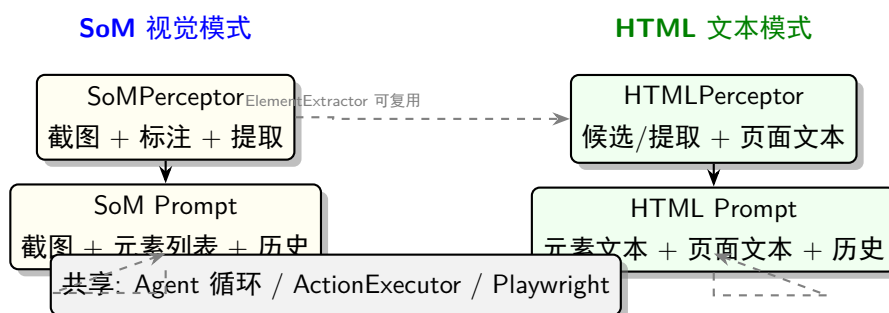


图 2: SoM 模式与 HTML 模式的架构对比

HTML 模式无需额外开发，仅需替换 Perceptor 和 Prompt 即可从视觉输入切换为文本输入，有效降低了为不同数据集定制评测流程的成本。

三 实验评测与分析

针对项目模型的 SoM 输入与 HTML 输入模式分别设计了评测实验。所有本地模型均在一台 RTX 3090 24GB $\times 1$, 内存 60GB 的计算机上运行, 并使用 Qwen 3VL-8B-Instruct 作为本地测试模型, 使用 API 调用 Qwen3.7-max 测试。选取任务成功率 (SR)、元素准确率 (Ele. Acc)、任务单步操作的 F1 分数 (Op.F1)、任务单步成功率 (Step SR) 以及单步平均耗时作为评价 Web Agent 性能的标准。其中, Ele. Acc 表示各任务内选择正确网页元素的步数比例跨任务的均值, Op.F1 分数表示单步操作的 F1 值, Step SR 表示各任务内元素与操作均正确的步数比例跨任务的均值, SR 表示所有步骤均成功执行的任务数占数据集总任务数的比例。

$$\begin{aligned} \text{Ele.Acc} &= \frac{1}{M} \sum_{i=1}^M \frac{1}{N_i} \sum_{j=1}^{N_i} \mathbb{I}(\hat{e}_{i,j} \in S_{acc}(e_{i,j})) \\ \text{Op.F1}_{i,j} &= \mathbb{I}(\hat{op}_{i,j} \in S_{op}(op_{i,j})) \times \begin{cases} 1 & op_{i,j} = \text{CLICK} \\ \text{F1}(v_{i,j}, \hat{v}_{i,j}) & op_{i,j} \in \{\text{TYPE}, \text{SELECT}\} \end{cases} \\ \text{Step SR} &= \frac{1}{M} \sum_{i=1}^M \frac{1}{N_i} \sum_{j=1}^{N_i} [\mathbb{I}(\hat{e}_{i,j} \in S_{acc}(e_{i,j})) \times \mathbb{I}(\text{Op.F1}_{i,j} = 1)] \\ \text{SR}_i &= \prod_{j=1}^{N_i} [\mathbb{I}(\hat{e}_{i,j} \in S_{acc}(e_{i,j})) \times \mathbb{I}(\text{Op.F1}_{i,j} = 1)] \end{aligned}$$

其中 M 为数据集内的总任务数, N_i 为第 i 个任务的步骤数, $e_{i,j}, \hat{e}_{i,j}$ 分别为第 i 个任务第 j 步中 Ground Truth 的等价网页元素集合和 Agent 预测的网页元素, $op_{i,j} \in \{\text{CLICK}, \text{TYPE}, \text{SELECT}\}$ 为 Ground Truth 操作类型, $\hat{op}_{i,j}$ 为预测操作类型, $v_{i,j}, \hat{v}_{i,j}$ 分别为 Ground Truth 和预测的操作值。 $S_{op}(\text{CLICK}) = \{\text{CLICK}, \text{HOVER}, \text{PRESS}\}$ 为 Mind2Web 中将 HOVER 和 PRESS_ENTER 映射至 CLICK 的等价操作集合; $\text{F1}(\cdot, \cdot)$ 为 F1 分数。

1. HTML 输入模式下的性能测评

选用 Mind2Web 数据集对模型进行评测。Mind2Web 数据集是一个大规模的 Web Agent 基准数据集, 将真实世界的静态网页作为数据, 具有 1,341 个不同任务和 46 步的最长任务步数, 在 Web Agent 领域被广泛用于测试模型的页面理解与执行能力。

我们对 Mind2Web 数据集提供的评测方法进行了修改。在 Mind2Web 数据集提供的 HTML 应用过滤规则只保留可交互元素后，使用一个基于 Bert2 实现的预训练模型 all-MiniLM-L6-v2 对剩余候选元素进行排序并交给 VLM 进行选择，并给出对应的操作类型和元素，最后计算 Ele. Acc, OP.F1, Step SR 和 SR。

表 1: HTML 输入模式下模型在 Mind2Web 数据集上的性能

	Cross-Task				Cross-Website				Cross-Domain				Time(s)
	EA	OP	SS	SR	EA	OP	SS	SR	EA	OP	SS	SR	
Qwen3-VL-8B	41.9	84.0	33.9	1.5	32.0	78.0	26.4	1.0	36.0	83.0	28.0	3.0	1.3
Qwen3.7-max	64.6	80.5	52.0	8.7	57.9	81.3	42.4	6.2	55.6	81.9	43.8	7.3	14.3

由表 1 可以看出，Qwen3.7-max 在元素定位上显著优于 Qwen3-VL-8B，元素准确率平均比 Qwen3-VL-8B 提高了 22.7%，最终导致更高的单步成功率与任务成功率。但 Qwen3-VL-8B 作为小模型具有快速响应与低成本的优势，在相对较好的网页操作性能和较快的响应时间中取得了平衡。Qwen3.7-max 的参数量过大，虽然准确率更高但响应时间是 Qwen3-VL-8B 的 11 倍，不具有实时性。

2. SoM 输入模式下的性能测评

为评估 SoM 视觉输入模式在真实网页场景下的表现，自行设计了包含 30 个任务的测试集，全部基于可公开访问的真实网站。测试任务覆盖多种难度与应用场景，代表性任务包括：搜索”中山大学广州校区南校园”并返回地址、打开哔哩哔哩首页点击第一个视频后返回作者头像、登录 QQ 邮箱并向指定地址发送端午祝福邮件等。完整任务列表见附录。所有测试均使用 Qwen 3VL-8B-Instruct 模型，在有头浏览器模式下运行，每个任务至少重复运行 10 次以减小随机性。

由于自建测试集缺少 Mind2Web 数据集中的 Ground Truth 标注（如真实操作序列和正负候选元素），无法直接计算 Ele. Acc 和 Op.F1 等微观指标。因此针对真实网页场景，定义以下两项宏观指标评价 Agent 的任务完成能力：

$$SR = \frac{S}{N}$$

$$AvgSteps = \frac{1}{S} \sum_{i \in S} steps_i$$

其中 $N = 30$ 为测试任务总数， S 为成功完成（Agent 返回正确答案且人工验证通过）的任务数， $\mathcal{S}$ 为成功任务集合， steps_i 为第 i 个任务成功完成时使用的步数。

表 2: SoM 输入模式与 HTML 输入模式在自建测试集上的对比

模型	SoM 输入		HTML 输入	
	SR	AvgSteps	SR	AvgSteps
Qwen 3VL-8B	73.3	10.5	40.0	6.8

由表 2 可以看出，SoM 视觉输入模式相比 HTML 文本输入模式在真实网页任务上具有显著优势。任务成功率从 40.0% 提升至 73.3%，提升幅度达 83.3%。分析任务完成情况发现，HTML 输入模式能够成功完成的任务以简单的搜索引擎查询为主（如事实类搜索、天气查询等），一旦任务涉及表单填写、页面跳转、多步交互等复杂操作，成功率显著下降。SoM 模式由于能够直观地识别页面布局与元素空间位置，在需要点击特定按钮、填写表单、切换标签页等复杂操作时表现更好。但 SoM 模式的完成任务平均步数 10.5 步明显高于 HTML 模式的 6.8 步，这一方面是因为 SoM 模式成功完成的任务本身更复杂，自然需要更多操作步数；另一方面也与 SoM 模式下模型倾向于执行更多探索性操作（如下拉滚动、尝试不同元素）有关。

总体而言，SoM 输入模式以更多的操作步数为代价，换取了更高的任务完成率与更广泛的任务适用性，在真实网页场景下优于 HTML 文本输入模式。

四 参考文献

参考文献

- [1] He, Y., et al. “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models.” 10.48550/arXiv.2401.13919
- [2] Microsoft Research. “Fara-7B: A Foundation Model for GUI Automation.” 2025.
- [3] Xinyi Li, et al. “A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges” Computer Science.10.1007/s44336-024-00009-2
- [4] Yang, J., et al. “Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V.” arXiv preprint arXiv:2310.11441, 2023.

- [5] Cheng, K., et al. “R-VLM: Region-Aware Vision Language Model for Precise GUI Grounding.” arXiv preprint 10.48550/arXiv.2507.05673.
- [6] Furuta, H., et al. “WebGUM: Multimodal Web Navigation with Instruction-Finetuned Foundation Models.” arXiv.2305.11854v4
- [7] Microsoft. “Playwright: Fast and Reliable Cross-Browser Automation.” <https://playwright.dev/>, 2025.
- [8] OpenClaw Contributors. “OpenClaw: Open-Source Multimodal Web Agent.” GitHub, 2025.
- [9] SMART-Lab Purdue. “SMART-LLM: Multi-Agent Task Planning.” GitHub, 2024.